МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І. МЕЧНИКОВА
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Євгенія ПЛАТОНОВА, Євген СТРАХОВ

Програмування

Основи програмування мовою Python. Структурне програмування

Конспект лекцій

Електронне навчальне видання

ОДЕСА
ОНУ
2026

УДК 004.4:004.421

ХХХ

Автори:

Є. В. Платонова, старший викладач кафедри оптимального керування і економічної кібернетики;

Є. М. Страхов, канд. фіз.-мат. наук, доцент, доцент кафедри оптимального керування і економічної кібернетики.

Рецензенти:

В. О. Сперанський, канд. техн. наук, доцент, доцент кафедри комп'ютеризованих систем та програмних технологій НУ «Одеська політехніка»;

З. Ю. Журавльова, доктор фіз.-мат. наук, доцент кафедри методів математичної фізики ОНУ імені І. І. Мечникова.

Рекомендовано до видання Методичною радою ОНУ імені І.І. Мечникова

(протокол № _____ від _____._____.2026 р.)

Платонова Є.В., Страхов Є.М.

ХХХ Основи програмування мовою Python. Структурне програмування [Електронний ресурс]: конспект лекцій для здобув. першого (бакалавр) рівня вищої освіти спец. Е7 "Математика"/ Є.В. Платонова, Є.М. Страхов – Електронні текстові дані (1 файл: 364 Кб). Одеса: Одес. нац. ун-т ім. І. І. Мечникова, 2026. – 55 с. ISBN ХХХ-ХХХ-ХХХ-ХХХ-Х

Конспект лекцій є вступом до базових принципів програмування мовою Python та систематизацією основних алгоритмічних конструкцій. У матеріалах конспекту розглядаються три базові конструкції керування виконанням програми, наведено приклади реалізації алгоритмів у Python. Конспект лекцій розроблений для студентів першого курсу спеціальності Е7 «Математика» та може бути використаний для самостійного навчання. Матеріал спрямований на формування алгоритмічного мислення та набуття практичних навичок програмування мовою Python.

УДК 004.4:004.421

ISBN ХХХ-ХХХ-ХХХ-ХХХ-Х

© Платонова Є.В., Страхов Є.М., 2026

© Одеський національний університет
імені І. І. Мечникова, 2026

Зміст

1 Програмування 6

1.1 Цілі курсу . . . 6

1.2 Що таке програмування? . . . 6

1.3 Структурне програмування . . . 7

2 Чому Python? Створення та запуск файлів. Робота в інтерпретаторі. 11

2.1 Історія та Глобальний вплив . . . 11

2.2 Переваги мови або чому Python . . . 12

2.3 Середовище для роботи та Інтерпретатор . . . 13

2.4 Виконання вказівок у Python . . . 14

2.5 Перевірка коду на помилки в IDLE . . . 15

3 Синтаксис Python. Типи Даних 17

3.1 Змінні та Ідентифікатори . . . 17

3.2 Зарезервовані слова Python . . . 17

3.3 Оператори та Вирази . . . 18

3.4 Основні Типи Даних і Операції . . . 19

3.5 Приклади. Виділення цифр числа . . . 21

3.6 Вбудовані функції для роботи з числами . . . 22

3.7 Оператори інкременту та особливості `float` . . . 23

4 Умовний оператор if 26

4.1 Оператори порівняння . . . 26

4.2 Синтаксичні форми умовного оператора . . . 27

5 Логічні оператори (and, or, not) 31

6 Генерація випадкових чисел: модуль random 35

7 Оператор циклу `for` та функція `range()` 38

7.1 Генератор послідовностей `range()` . . . 38

7.2 Базові алгоритми з циклом `for` 39
7.3 Керування циклом: `break`, `continue` та `else` 43

8 Оператор циклу `while` 46
8.1 Керування циклом: `break`, `continue`, `pass` та блок `else` 47
8.2 Типові застосування циклу `while` 48
8.3 Проблема «зациклювання» 50
8.4 Складніші приклади та алгоритми 52

Вступ

Дисципліна «Програмування» є базовою складовою підготовки студентів спеціальності Е7 «Математика» та формує фундаментальні знання і навички, необхідні для подальшого застосування сучасних інформаційних технологій у наукових дослідженнях та професійній діяльності. Вона забезпечує розвиток алгоритмічного мислення, вміння будувати та аналізувати алгоритми, а також практичні навички роботи з мовами програмування.

Особливе місце у курсі посідає мова Python, яка завдяки простоті синтаксису, універсальності та широкому спектру бібліотек стала одним із ключових інструментів у сфері аналізу даних, машинного навчання та Data Science. Для студентів-математиків це відкриває можливості поєднання теоретичних знань із практичним застосуванням у статистиці, моделюванні, обробці великих масивів даних та побудові прогнозних моделей.

Основними завданнями курсу є:

- ☑ ознайомлення студентів з базовими алгоритмічними конструкціями структурного програмування;
- ☑ формування навичок написання програм мовою Python;
- ☑ розвиток умінь застосовувати програмування для розв’язання математичних задач;
- ☑ підготовка до використання Python у подальших дисциплінах, пов’язаних з аналізом даних та Data Science.

Таким чином, курс «Програмування» є важливим етапом у професійній підготовці студентів спеціальності Е7 «Математика», оскільки створює основу для подальшого опанування сучасних методів аналізу даних, статистичних моделей та технологій штучного інтелекту.

Цей конспект лекцій охоплює лише частину курсу з програмування мовою Python і зосереджений на концепціях структурного програмування. Питання, пов’язані зі складнішими структурами даних, залишаються поза межами цього видання та розглядаються в наступних частинах курсу.

Програмування

1

1.1 Цілі курсу

- ☑ навчитися складати алгоритми;
- ☑ навчитися писати фрагменти коду (текст програми);
- ☑ навчитися читати чужий код і використовувати його.

1.2 Що таке програмування?

Слово «програма» походить від грецьких слів «pro» (перед) і «graphein» (писати). Іншими словами, це був акт написання чогось наперед або достроково.

Програмування — це

процес проєктування, написання, тестування, впровадження і підтримки комп'ютерних програм. У вужчому значенні програмування розглядається як кодування — реалізація у вигляді програми одного чи кількох взаємопов'язаних алгоритмів.

У широкому сенсі процес програмування охоплює і створення, тобто розробку алгоритмів, і аналіз потреб майбутніх користувачів програмного забезпечення. Технологія програмування постійно розвивається, що призводить до появи нових мов програмування, інструментів розробки і сфер застосування.

Алгоритм — це

скінченна послідовність вказівок на виконання дій, спрямованих на розв'язання певної задачі.

Основні властивості алгоритму

Скінченність — алгоритм складається з обмеженої кількості кроків і має завершуватися після виконання.

Визначеність (детермінованість) — кожна команда алгоритму повинна бути чіткою й однозначною, без двозначності чи різних тлумачень.

Масовість (загальність) — алгоритм застосовується для широкого класу задач або різних вхідних даних, а не лише для одного конкретного випадку.
Результативність — виконання алгоритму повинно приводити до розв'язання задачі або отримання результату.
Дискретність — алгоритм складається з окремих кроків (операцій), які виконуються послідовно.

1.3 Структурне програмування

Структурне програмування — це методологія розробки програм, яка ґрунтується на використанні трьох базових конструкцій керування виконанням:

- ☑ слідування (лінійний алгоритм),
- ☑ розгалуження (умовний оператор),
- ☑ цикл (повторення).

Слідування (лінійний алгоритм)

Лінійні алгоритми — це

алгоритми, в яких дії виконуються послідовно одна за одною.

Це означає, що кожна команда лінійного алгоритму виконується строго в заданому порядку. Лінійні алгоритми можуть бути корисними для розв'язання простих задач, таких як знаходження суми двох чисел або переведення одиниць виміру. Наприклад, лінійний алгоритм для знаходження суми двох чисел може мати такий вигляд:

Приклад: обчислення суми двох чисел.

Алгоритм: Сума двох чисел

Ввести a, b
Обчислити $s = a + b$
Вивести s

Розгалуження (умовний оператор)

Розгалуження — це

алгоритмічна конструкція, яка дає змогу змінювати хід виконання програми залежно від істинності певної умови.

Умовний оператор — це

конкретна реалізація розгалуження у мові програмування. Він перевіряє логічний вираз і, залежно від його значення (true або false), виконує одну з гілок коду.

Приклад: перевірка парності числа.

Алгоритм: Перевірка парності

Ввести n
if n mod $2 = 0$ **then**
  Вивести “Число парне”
else
  Вивести “Число непарне”
end if

Цикл (повторення)

Цикл — це

алгоритмічна конструкція, яка забезпечує багаторазове виконання одних і тих самих дій (команд) доти, доки виконується певна умова або поки не буде досягнуто заданої кількості повторень.

Цикли дають змогу:

- ☑ автоматизувати повторювані обчислення;
- ☑ скоротити код, замінивши багаторазове дублювання;
- ☑ будувати алгоритми для обробки масивів даних, серій чисел, введення/виведення.

Приклад: обчислення суми чисел від 1 до n.

Алгоритм: Сума чисел від 1 до n

Ввести n
$s := 0$
for $i := 1$ **to** n **do**
  $s := s + i$
end for
Вивести s

Теорема структурного програмування

У 1970□х роках Едсгер Дейкстра та Ніклаус Вірт сформулювали принципи структурного програмування. Вони довели, що

будь-який алгоритм можна подати як композицію трьох базових блоків: слідування, розгалуження, цикл.

Логічна повнота конструкцій: слідування забезпечує лінійність і впорядкованість дій, розгалуження додає можливість ухвалювати рішення (умовний вибір), цикл дає змогу повторювати дії довільну кількість разів. Разом ці три конструкції утворюють повний набір структур керування, достатній для опису будь-якого алгоритму.

Усі сучасні мови програмування (Python, C++, Java) мають оператори для цих трьох конструкцій (if, for, while).

Навіть складні алгоритми (наприклад, сортування, пошук, обробка даних) можна розкласти на комбінацію цих трьох структур.

Підпрограми (функції, процедури) лише групують ці базові блоки, але не додають нових типів керування.

Контрольні питання

1. Що таке програмування? Які основні етапи воно містить?
2. Дайте означення алгоритму. Які його основні властивості?
3. Що таке структурне програмування? Які три базові конструкції воно використовує?
4. Поясніть, що таке слідування (лінійний алгоритм). Наведіть приклад.
5. Що таке розгалуження? Чим воно відрізняється від умовного оператора?

6. Наведіть приклад умовного оператора у будь-якій мові програмування.
7. Що таке цикл? Які основні види циклів ви знаєте?
8. Наведіть приклад алгоритму з використанням циклу.
9. Сформулюйте теорему структурного програмування. На чому ґрунтується її доведення?
10. Чому саме три конструкції (слідування, розгалуження, цикл) вважаються достатніми для побудови будь-якого алгоритму?

Чому Python? Створення та запуск файлів. Робота в інтерпретаторі.

2

2.1 Історія та Глобальний вплив

Створення Python розпочав Гвідо ван Россум (Guido van Rossum) у 1991 році під час роботи над розподіленою ОС «Амеба». Йому була потрібна розширювана мова, яка б забезпечувала підтримку системних викликів. За основу було взято мови ABC та Modula-3. Назву Гвідо обрав на честь популярного комедійного шоу BBC «Літаючий цирк Монті-Пайтона», а не на честь змії. Відтоді Python активно розвивався за підтримки організацій, у яких працював його автор. Сьогодні мова вдосконалюється ще швидше, адже над нею працює не лише команда творців, а й велика міжнародна спільнота програмістів. Втім, остаточне слово щодо напряму розвитку Python і надалі залишається за Гвідо ван Россумом.

Історія розвитку Python

- **Python 2.0** — випущений у 2000 році. Серед нововведень: спискові включення, підтримка Unicode, вдосконалений збирач сміття.
- **Python 3.0** — випущений у 2008 році. Основна особливість — несумісність із попередніми версіями, що вимагало адаптації бібліотек.
- **Python 2.7** — останній реліз гілки 2.x. Його підтримку офіційно завершено у 2020 році, що ознаменувало перехід до епохи Python 3.

Глобальний вплив та переваги

Python став однією з найпопулярніших мов програмування у світі завдяки простоті синтаксису та універсальності. Його активно використовують такі компанії, як **Google, Netflix, Facebook, Instagram, Spotify** та багато інших.

Основні сфери застосування:

- **Наука та дослідження** — бібліотеки NumPy, SciPy, pandas.

- **Машинне навчання та штучний інтелект** — TensorFlow, PyTorch, scikit-learn.
- **Веб-розробка** — фреймворки Django, Flask.
- **Автоматизація та DevOps** — скрипти для адміністрування, тестування, CI/CD.
- **Аналіз даних та візуалізація** — matplotlib, seaborn, Jupyter Notebooks.

Таким чином, Python використовується практично всюди — від навчальних проєктів до масштабних систем світових технологічних гігантів.

2.2 Переваги мови або чому Python

Основні переваги

- **Якість програмного забезпечення.** Зручність читання, ясність і висока якість відрізняють Python від інших мов. Єдиноманітність оформлення коду полегшує його розуміння навіть для тих, хто не брав участі у створенні програми.
- **Висока швидкість розробки.** Порівняно з C, C++ та Java, Python значно підвищує продуктивність праці розробника. Обсяг коду зазвичай становить третину або навіть п'яту частину еквівалентного коду на C++ чи Java. Програми запускаються відразу, минаючи етапи компіляції та зв'язування.
- **Перенесення програмного коду без витрат.** Більшість програм виконується без змін на основних платформах. Перенесення з Linux у Windows часто зводиться до копіювання файлів. Python також підтримує створення графічних інтерфейсів, доступ до баз даних, вебзастосунків та інших типів програм.
- **Бібліотеки підтримки.** У складі Python є велика стандартна бібліотека, що охоплює пошук тексту, роботу з мережею та інші функції. Крім того, існують численні сторонні бібліотеки: для веб-розробки, математичних обчислень, ігор тощо. Наприклад, `NumPy` — потужний інструмент для наукових обчислень.
- **Інтеграція компонентів.** Python легко взаємодіє з іншими мовами та платформами: може викликати функції з бібліотек на C/C++, інтегруватися з Java, взаємодіяти з COM та .NET, працювати через протоколи SOAP, XML-RPC, CORBA.
- **Вільне використання.** Python можна використовувати та розповсюджувати абсолютно безкоштовно.

Класифікація мови

1. **Високого рівня.** Код легко читати і писати, він схожий на англійську мову.
2. **Загального призначення.** Python розв'язує широкий спектр завдань.
3. **Інтерпретована.** Для виконання коду не потрібна компіляція.

Python часто позиціонують як **об'єктно-орієнтовану мову сценаріїв**, що дає змогу створювати прості програми у вигляді одного файлу, використовуючи об'єкти.

Недоліки

- ☑ **Швидкодія.** Програми на Python можуть працювати повільніше за аналоги, написані компільованими мовами (наприклад, C).
- ☑ **Байт-код.** Python перетворює вихідний код у байт-код, який інтерпретується. Це забезпечує переносність програм, але відсутність машинного коду може знижувати продуктивність.

2.3 Середовище для роботи та Інтерпретатор

Python — це не лише мова програмування, а й програмний пакет, що містить **інтерпретатор** — модуль, який створює та виконує інші програми.

Онлайн-середовища розробки

Сьогодні є можливість писати код на Python у веб-сервісах, які дають змогу запускати програми прямо у браузері, без встановлення додаткового програмного забезпечення:

- ☑ https://replit.com/~
- ☑ https://ideone.com/
- ☑ https://www.online-python.com/

У цих сервісах достатньо створити акаунт, написати код та запустити його онлайн.

Командний рядок інтерпретатора

Найпростіший спосіб писати й запускати програми на Python — вводити інструкції безпосередньо у **командний рядок інтерпретатора**. Це дає змогу швидко

перевіряти окремі команди та експериментувати з кодом.

Локальне встановлення Python

Щоб працювати локально, потрібно завантажити Python з офіційного сайту:

- ☑ https://www.python.org/downloads/

Після встановлення пакета Python створюється інтерпретатор та бібліотека підтримки. Також доступне інтегроване середовище розробки **IDLE**, яке встановлюється разом із Python і дає змогу писати, запускати та налагоджувати програми у зручному графічному інтерфейсі.

2.4 Виконання вказівок у Python

Інтерактивний режим

В інтерактивному режимі користувач вводить інструкції та натискає `Enter`, після чого результат відображається рядком нижче.

```
>>> 2 + 2
4
>>> "Hello"
'Hello'
```

Інтерактивний режим дає змогу:

- ☑ швидко перевіряти фрагменти коду;
- ☑ проводити експерименти без створення файлів;
- ☑ одразу виявляти помилки.

Приклад помилки:

```
>>> print(y)
Traceback (most recent call last):
File "<pyshell#15>", line 1, in <module>
print(y)
NameError: name 'y' is not defined
```

Виведення повідомлень на екран

Для виведення інформації використовується команда `print`:

```
>>> print("Hello World")
Hello World
>>> print(2 + 2)
4
```

В інтерактивному режимі можна вводити вирази без `print`, але у файлах інструкцію `print` потрібно писати обов'язково.

Програми у файлах

Програму на Python можна написати у будь-якому текстовому редакторі та зберегти у файл з розширенням `.py`. Наприклад, створимо файл `hello.py`:

```
print("Hello world!")
print("Привіт, світе!")
input()
```

Запуск програми здійснюється подвійним клацанням на файлі. Такі програми виконуються у консольному вікні («чорному вікні»), де введення здійснюється з клавіатури. Якщо у програмі є помилки, у консолі буде показано повідомлення про них.

Таким чином, Python дає змогу працювати як в інтерактивному режимі, так і створювати повноцінні програми у файлах.

2.5 Перевірка коду на помилки в IDLE

Запуск програми у IDLE

1. Запускаємо середовище **IDLE**.
2. У меню **File** обираємо пункт **Open** та у вікні шукаємо файл `hello.py`.
3. Відкриється нове вікно з текстом програми. У цьому вікні у меню **Run** вибираємо **Run Module** або натискаємо клавішу `F5`.

Результати виконання

- ☑ Якщо у програмі немає помилок, введення та виведення здійснюється у вікні інтерпретатора.
- ☑ Повідомлення про помилки також відображаються у **IDLE**.
- ☑ Якщо програма працює некоректно, але не видає помилок, для пошуку проблем можна вставити додаткові інструкції `print`.

Приклад використання `print` для налагодження

```
print("Значення змінної x:", x)
```

Це допоможе зрозуміти, які значення мають змінні під час виконання програми.

Контрольні питання

1. Які основні сфери застосування Python ви знаєте?
2. Назвіть головні переваги мови Python.
3. Які недоліки Python були згадані у тексті?
4. Як класифікується мова Python (за рівнем, призначенням, способом виконання)?
5. Що таке інтерпретатор Python і яку роль він виконує?
6. Які онлайн□середовища можна використати для написання та запуску програм на Python?
7. Як працювати з Python у командному рядку інтерпретатора?
8. Які кроки потрібно виконати для локального встановлення Python?
9. Чим відрізняється інтерактивний режим від запуску програм у файлах?
10. Як у Python здійснюється виведення повідомлень на екран?
11. Як перевіряти код на помилки у середовищі IDLE?
12. Які результати виконання програми можна побачити у IDLE?

Синтаксис Python. Типи Даних

3

3.1 Змінні та Ідентифікатори

Змінна - це

ім'я (ідентифікатор), яке може набувати певного значення.

У Python змінні не потрібно оголошувати, значення можна надати одразу за допомогою символу =.

Приклади присвоєння

```
x = 4
s1 = "Hello"
_x = True
x2 = 23.46
summa1 = summa2 = k = 0
```

Ідентифікатор - це

унікальна ознака об'єкта, що дозволяє розрізняти його з іншими.

Правила побудови ідентифікаторів

- ☑ Використовуються лише латинські літери, цифри та знак підкреслення (_).
- ☑ Ідентифікатор повинен починатися з літери або символу підкреслення.
- ☑ Регістр літер має значення (`X` ≠ `x`).
- ☑ Неприпустимі пробіли, точки, спеціальні символи та літери кирилиці.
- ☑ Не можна застосовувати спеціальні символи `$`, `%`, `@`.
- ☑ Заборонено використовувати зарезервовані слова Python (`if`, `for`, `while`, `class`, `def` тощо).

3.2 Зарезервовані слова Python

У Python існує набір **ключових слів**, які не можна використовувати як ідентифікатори (назви змінних, функцій, класів тощо). Ці слова мають спеціальне значення

у синтаксисі мови.

```
False   class     finally  is       return
None    continue  for      lambda   try
True    def       from     nonlocal while
and     del       global   not      with
as      elif      if       or       yield
assert  else      import   pass     break
```

Використання будь-якого з цих слів як назви змінної або функції призведе до помилки.

Приклад помилки

```
>>> class = 5
File "<stdin>", line 1
class = 5
^
SyntaxError: invalid syntax
```

3.3 Оператори та Вирази

Вираз — це

фрагмент мови програмування, що представляє спосіб обчислення деякого значення.

Операнд — це

частина виразу, над яким виконуються операції.

Оператор — це

дія, яка може бути виконана над одним або декількома операндами.

Приклади синтаксису

```
x=4+5 % Присвоєння
print(x)
s="Hello "+"World"
print(s)
s="Helo"+\
"World" % Перенесення довгих операторів
```

Правила форматування коду

- ☑ Кожен оператор завершується символом нового рядка.
- ☑ Відступи застосовуються для позначення різних блоків коду (тіла функцій, умовні висловлювання, цикли).
- ☑ Пробіли неприпустимі перед операторами, але необхідні перед вкладеними блоками (`unexpected indent`).
- ☑ Якщо потрібно розмістити кілька операторів в одному рядку, їх розділяють крапкою з комою (`;`).
- ☑ Символ # означає коментар.
- ☑ Багаторядковий коментар укладають у три подвійні лапки в `py` файлі.

3.4 Основні Типи Даних і Операції

Вбудовані типи даних:

- ☑ Числа: `1234`, `3.14`, `3+4j`.
- ☑ Рядки: `'spam'`, `"guido's"`.
- ☑ Списки: `[1, [2, 'three'], 4]`.
- ☑ Словники: `{'food': 'spam', 'taste': 'yum'}`.
- ☑ Кортежі: `(1, 'span', 4, 'U')`.
- ☑ Файли: `myfile = open('eggs', 'r')`.
- ☑ Множини: `set('abc')`, `{'a', 'b', 'c'}`.

Вбудовані числові типи

- ☑ цілі числа (числа без дробової частини) із необмеженою точністю (`3`, `3465`, `45`, `42`, `-4`);
- ☑ дійсні числа (числа з рухомою або плавальною крапкою): `3.14`, `5.68`;

- ☑ вісімкові числа (починаються з префікса `0o`): `0o77 = 63`, `0o10 = 8`;
- ☑ шістнадцяткові числа (починаються з префікса `0x`): `0xFF = 255`, `0xff = 255`;
- ☑ двійкові числа (починаються з префікса `0b`): `0b1111 = 15`, `0b10 = 2`;
- ☑ комплексні числа з уявною частиною: `3+4j`, `3+4J` (за допомогою великої або малої літери `j`).

Символ множення перед j не вказується.

```
>>>z1=2+3j
>>>z2=4+8J
>>>print(z1, z2)
(2+3j) (4+8j)
```

Комплексні числа можуть бути створені за допомогою функції `complex(дійсна_частина, уявна_частина)`

```
>>>z3=complex(34,2)
>>>print(z3)
(34+2j)
```

Операції з числовими типами

Оператор	Опис	Приклад	Результат
+	Додавання	5+7	12
-	Віднімання	55-10	45
*	Множення	4*6	24
/	Ділення	7 / 2	3.5
//	Цілочислове ділення	7 // 2	3
%	Остача від ділення	7 % 3	1
**	Піднесення до степеня	3 ** 4	81

Результатом операції буде новий об’єкт.

```
2 ** 100 # 2 у степені 100
```

1267650600228229401496703205376

Зверніть увагу на результат останньої операції: у Python 3 цілі числа автоматично забезпечують **необмежену** точність. Наприклад, ви можете спробувати обчислити 2 у степені 1 000 000 (але навряд чи варто це робити, оскільки на екран

буде виведено число завдовжки понад 300 000 знаків, що може забрати тривалий час!).

> len(str(2 ** 1000000))

Скільки цифр у дійсно великому числі
301030

3.5 Приклади. Виділення цифр числа

Двоцифрове число

Нехай маємо число `n = 47`. Щоб виділити окремі цифри:

Двоцифрове число

```
n = 47
d1 = n // 10    # перша цифра (десятки)
d2 = n % 10     # друга цифра (одиниці)
print("Десятки =", d1, "Одиниці =", d2)
```

Результат:

```
Десятки = 4 Одиниці = 7
```

Трицифрове число

Нехай маємо число `m = 583`. Щоб виділити окремі цифри:

Трицифрове число

```
m = 583
sot = m // 100           # сотні
des = (m // 10) % 10     # десятки
od = m % 10              # одиниці
print("Сотні =", sot, "Десятки =", des, "Одиниці =", od)
```

Результат:

```
Сотні = 5 Десятки = 8 Одиниці = 3
```

Пріоритет операторів - порядок виконання тут

3.6 Вбудовані функції для роботи з числами

Python надає низку **вбудованих функцій**, які можна застосовувати до числових типів.

Функція `abs(a)`

Повертає модуль числа.

Приклад

```
x = -12
print(abs(x))    # 12
```

Функція `divmod(a, b)`

Виконує ділення `a` на `b` та повертає пару чисел: частку і остачу.

Приклад

```
x = 23
y = 7
print(divmod(x, y))    # (3, 2)
```

Функція `hex(x)`

Конвертує ціле число у рядок із шістнадцяткових цифр.

Приклад

```
x = 15
print(hex(x))    # '0xf'
```

Функція `oct(x)`

Конвертує ціле число у рядок із вісімкових цифр.

Приклад

```
x = 15
print(oct(x))    # '0o17'
```

Функція `round(x)`

Округлює число до найближчого цілого.

Приклад

```
print(round(3.0254))    # 3
```

Функція `round(x, n)`

Округлює число `x` з точністю до `n` знаків після коми.

Приклад

```
print(round(3.0254, 2))    # 3.03
```

3.7 Оператори інкременту та особливості `float`

У Python відсутні оператори `++` та `--`. Замість них використовуються комбіновані оператори присвоєння:

- ☑ `x += 1` замість `x = x + 1`
- ☑ `x -= 1` замість `x = x - 1`
- ☑ `x *= 2` замість `x = x * 2`

Обережно з дійсними числами (float)!

Числа з плавальною (рухомою) крапкою зберігаються в пам'яті комп'ютера у двійковій системі з обмеженою точністю (53 біти). Через це обчислення можуть містити незначну похибку:
0.1 + 0.1 + 0.1 буде дорівнювати 0.30000000000000004
Для порівняння дійсних чисел завжди використовуйте функцію `math.isclose(a, b)` або порівняння з точністю ε: `abs(a - b) < 1e-9`.

Контрольні питання

1. Що таке змінна у Python? Як їй надається значення?
2. Що таке ідентифікатор? Які правила його побудови?

3. Які символи та слова заборонено використовувати як ідентифікатори?
4. Що таке зарезервовані слова у Python? Наведіть приклади.
5. Що станеться, якщо використати зарезервоване слово як назву змінної?
6. Дайте означення виразу, операнда та оператора.
7. Які правила форматування коду у Python ви знаєте?
8. Як у Python позначають коментарі? Чим відрізняється однорядковий від багаторядкового коментаря?
9. Назвіть основні вбудовані типи даних у Python.
10. Які числові типи даних підтримує Python?
11. Як у Python записуються цілі, дійсні, вісімкові, шістнадцяткові та двійкові числа?
12. Як створюються комплексні числа у Python? Наведіть приклад.
13. Які основні арифметичні оператори підтримує Python? Наведіть приклади їх використання.
14. Що означає оператор цілочислового ділення та оператор остачі від ділення?
15. Які вбудовані функції для роботи з числами ви знаєте? Наведіть приклади їх використання.
16. Як працює функція `round()`? Чим відрізняється `round(x)` від `round(x, n)`?
17. Як можна виділити окремі цифри з двоцифрового чи трицифрового числа?

Практичні завдання

1. Створіть змінну `x` та надайте їй значення 25. Виведіть її на екран.
2. Складіть програму, яка перевіряє, чи є число парним або непарним.
3. Напишіть програму, яка виділяє десятки та одиниці з двоцифрового числа.
4. Напишіть програму, яка виділяє сотні, десятки та одиниці з трицифрового числа.
5. Обчисліть результат виразу: `(15 // 4) + (15 % 4)`. Поясніть отриманий результат.
6. Створіть дві змінні `z1` та `z2` як комплексні числа. Виконайте їх додавання та виведіть результат.
7. Використайте функцію `abs()` для знаходження модуля від'ємного числа.
8. Використайте функцію `divmod()` для ділення числа 29 на 5. Виведіть частку та остачу.

9. Використайте функцію `round()` для округлення числа 3.14159 до двох знаків після коми.
10. Напишіть програму, яка обчислює кількість цифр у числі 2^{1000}.

Умовний оператор if

4

За замовчанням хід виконання програми є лінійним: інструкції виконуються послідовно — рядок за рядком, від першого до останнього. Послідовне виконання є базою, але реальні алгоритми вимагають гнучкості та адаптивності.

У більшості задач програма повинна ухвалювати рішення залежно від вхідних даних чи поточного стану обчислень. Для цього використовуються розгалуження (або умовні переходи).

Оператор if в Python — це

умовний оператор, який дає змогу програмі ухвалювати рішення та виконувати певний блок коду лише тоді, коли виконується задана умова

В основі ухвалення рішень у програмі лежать оператори порівняння (відношення). Вони дають змогу зіставляти значення виразів і повертають булеве значення — **True (істина)** або **False (хибність)**.

4.1 Оператори порівняння

Python підтримує базові математичні операції відношення:

```
Дорівнює:                a == b
Не дорівнює:             a != b
Менше ніж:               a < b
Менше або дорівнює:     a <= b
Більше ніж:              a > b
Більше або дорівнює:    a >= b
```

Зверніть увагу:

Оператор перевірки на **рівність** позначено двома знаками ==, оскільки один знак = в Python зарезервовано для операції **присвоєння**.

На відміну від багатьох інших мов програмування, Python дає змогу використовувати ланцюгові порівняння (подвійні нерівності), що звично для математиків, наприклад: **a < x <= b** замість запису через логічне «і».

Отримані булеві вирази є фундаментом для керування потоком виконання про-

грами: вони використовуються в умовних конструкціях (if, elif, else) та циклах (while).

4.2 Синтаксичні форми умовного оператора

Залежно від кількості напрямків (гілок) виконання програми, в Python застосовують три основні форми розгалуження:

1. Неповна форма (коротке розгалуження)

Використовується, коли певну дію потрібно виконати **тільки** за умови, що вираз-умова є істинним (`True`). Якщо умова хибна (`False`), блок коду просто пропускається.

```
Неповна форма
if вираз_умова1:
    оператор1
    оператор2
```

2. Повна форма (двогілкове розгалуження)

Застосовується, коли є два альтернативних шляхи: один виконується при істинності умови (`True`), а інший (блок `else`) — якщо умова хибна (`False`).

```
Повна форма
if вираз_умова1:
    оператор1
    оператор2
else:
    Блок_операторів_3
```

3. Багатогілкова форма (багатоальтернативне розгалуження)

Використовується для перевірки серії послідовних умов за допомогою конструкцій `elif` (скорочення від *else if*). Умови перевіряються зверху вниз: виконується блок першої умови, яка виявиться істинною, після чого весь оператор `if` завершує роботу.

```
Багатогілкова форма
if вираз_умова1:
    Блок_операторів_1
elif вираз_умова2:
    Блок_операторів_2
elif вираз_умова_n:
    Блок_операторів_n
else:
   Блок_операторів_n+1
```

Важливі синтаксичні особливості Python:

☑ **Двокрапка (:):** є обов'язковою в кінці кожного рядка з `if`, `elif` та `else`. Вона позначає початок відповідного блоку коду.

Приклад 1. Користувач вводить число та отримує відповідь, яке воно: додатне, від'ємне або нуль.

```
x=int(input("Введіть x "))
if x>0:
    print("x додатне")
else:
    print("x від'ємне або нуль")
    input()
```

Приклад 2. Порівняння двох значень (чи більше перше за друге або вони рівні)

```
A=int(input())
B=int(input())
if A>B: print("A більлше") #можна писати у тому ж рядку 1 оп-р
elif A==B: print("A дорівнює B")
else: print("A менше B")
```

Приклад 3. Користувач отримує відповідь, чи оплачений чек (звернути увагу на імена змінних)

```
print ("Привіт")
price1 = 50 #price1=int(input())
price2 = 32 #price2=int(input())
if price1+ price2 > 99 :
    print ("Сума недостатня")
else:
    print ("Чек оплачений")
    print ("До побачення")
```

Приклад 4. Вивести на екран місяці відповідної пори року

```
s=input("Введіть пору року")
if s=="зима":
    print("грудень")
    print("січень")
    print("лютий")
elif s=="весна":
    print("березень")
    print("квітень")
    print("травень")
elif s=="літо":
    print("червень")
    print("липень")
    print("серпень")
else:
    print("вересень")
    print("жовтень ")
    print("листопад")
print ("До побачення")
```

Контрольні питання

1. Що таке умовний оператор `if` у Python? Для чого він використовується?
2. Які оператори порівняння підтримує Python? Наведіть приклади.
3. Чим відрізняється оператор присвоєння = від оператора перевірки на рівність ==?
4. Що таке ланцюгові порівняння у Python? Наведіть приклад.
5. Які основні синтаксичні форми умовного оператора існують у Python?

6. У чому полягає відмінність між неповною та повною формою умовного оператора?
7. Як працює багатогілкова форма умовного оператора з `elif`?
8. Яка роль двокрапки `:` у синтаксисі умовних операторів?
9. Що відбувається, якщо умова у операторі `if` є хибною?
10. Як можна записати умовний оператор у одному рядку?

Практичні завдання

1. Реалізуйте програму, яка за двома введеними числами перевіряє, чи можна знайти частку від ділення першого на друге.
2. Напишіть програму, яка за введеними координатами точки визначає чверть.
3. Складіть програму, яка визначає найбільше з трьох введених чисел.
4. Реалізуйте програму, яка перевіряє, чи введене число є парним або непарним.
5. Напишіть програму, яка визначає, чи введене число кратне 10.
6. Складіть програму, яка за введеним віком користувача виводить повідомлення: «дитина», «підліток», «дорослий», «пенсіонер».
7. Реалізуйте програму, яка перевіряє, чи введений рік є високосним.
8. Напишіть програму, яка за введеною оцінкою (від 1 до 12) виводить текстове пояснення («незадовільно», «задовільно», «добре», «відмінно»).

Логічні оператори (and, or, not)

5

Прості умовні вирази (операції відношення) можна об'єднувати у складні **складені умови** за допомогою **логічних операторів** (булевих зв'язок). В Python використовуються три базові логічні оператори: **`and`, `or` та `not`**.

1. Кон'юнкція (`and` — логічне «І»)

Оператор **`and`** повертає **`True`** тоді й лише тоді, коли **обидва** операнди є істинними. Якщо хоча б один із виразів хибний (**`False`**), увесь складений вираз набуває значення **`False`**.

Пріоритет оператора **`and`** нижчий за пріоритет операторів порівняння (>, <, == тощо), тому у виразах на зразок:

Перевірка діапазону

```
if length > 8 and length < 20:
    print("Довжина відповідає вимогам")
```

дужки навколо окремих порівнянь писати не обов'язково.

2. Диз'юнкція (`or` — логічне «АБО»)

Оператор **`or`** повертає **`True`**, якщо **хоча б один** із операндів є істинним (або обидва одночасно). Вираз набуває значення **`False`** лише тоді, коли всі складові частини є хибними.

Приклад диз'юнкції

```
if age < 18 or age >= 60:
    print("Пільгова категорія")
```

3. Заперечення (`not` — логічне «НЕ»)

Унарний оператор **`not`** інвертує (змінює на протилежне) значення булевого виразу: **`True`** перетворюється на **`False`**, а **`False`** — на **`True`**.

Приклад заперечення

```
if not is_valid:
    print("Помилка введення даних")
```

Таблиця 5.1: Таблиця істинності для логічних операцій

Значення операндів		Результат операції		
X	Y	not X	X and Y	X or Y
False	False	True	False	False
False	True	True	False	True
True	False	False	False	True
True	True	False	True	True

Особливості Python:

- **Пріоритет виконання:** Найвищий пріоритет має `not`, далі йде `and`, і найнижчий — `or`. Для зміни порядку обчислення використовують круглі дужки `()`.
- **Ліниві обчислення (Short-circuit evaluation):** Python обчислює логічні вирази зліва направо і зупиняється, як тільки результат стає однозначним (наприклад, якщо у виразі з `and` перший операнд `False`, другий операнд навіть не обчислюється).

Приклад 1. Перевірка діапазону числа

```
x = int(input("Введіть число: "))
if x >= 10 and x <= 20:
    print("Число входить у діапазон від 10 до 20")
else:
    print("Число поза діапазоном")
```

Приклад 2. Визначення типу трикутника (рівнобедрений)

```
a = int(input("Введіть сторону a: "))
b = int(input("Введіть сторону b: "))
c = int(input("Введіть сторону c: "))

if a == b or a == c or b == c:
    print("Трикутник є рівнобедреним")
else:
    print("Трикутник не є рівнобедреним")
```

Приклад 3. Використання заперечення

```
password = input("Введіть пароль: ")
if not password:
    print("Пароль не може бути порожнім!")
else:
    print("Пароль прийнято")
```

Приклад 4. Перевірка: чи число належить до множини парних чисел від 10 до 20

```
x = int(input("Введіть число: "))
if (x % 2 == 0) and (x >= 10 and x <= 20):
    print("Число належить до множини парних чисел
     у діапазоні [10;20]")
```

Контрольні питання

1. Які логічні оператори підтримує Python? Для чого вони використовуються?
2. Що таке кон'юнкція (`and`)? Коли вираз з `and` повертає істину?
3. Що таке диз'юнкція (`or`)? У яких випадках вираз з `or` є істинним?
4. Як працює оператор заперечення (`not`)?
5. Який порядок пріоритетів виконання логічних операторів у Python?
6. Що таке «ліниві обчислення» (short-circuit evaluation)? Наведіть приклад.
7. Як можна комбінувати логічні оператори з операторами порівняння?
8. Наведіть приклад використання `and` для перевірки належності числа до діапазону.
9. Наведіть приклад використання `or` для перевірки кількох альтернативних умов.
10. Як можна використати `not` для перевірки, що число не входить у певний інтервал?
11. Що таке таблиця істинності? Як вона допомагає зрозуміти роботу логічних операторів?
12. Чому дужки важливі при комбінуванні кількох логічних операторів у складних виразах?

Практичні завдання

1. Напишіть програму, яка перевіряє, чи введене число належить до діапазону від 100 до 200 включно.
2. Напишіть програму, яка перевіряє, чи трикутник є рівнобедреним (дві сторони рівні), а не рівностороннім.
3. Складіть програму, яка перевіряє, чи число є парним і належить до діапазону від 50 до 100.
4. Реалізуйте програму, яка перевіряє, чи введене число є кратним 3 або 7.
5. Складіть програму, яка визначає, чи введене число є парним або кратним 5.
6. Реалізуйте програму, яка перевіряє, чи введене число належить до множини парних чисел у діапазоні [10;20].

Генерація випадкових чисел: модуль random

6

При розв'язанні багатьох математичних задач, моделюванні ймовірнісних процесів (метод Монте-Карло) або створенні ігор виникає потреба у використанні **випадкових чисел**. У Python для цього призначений вбудований модуль `random`.

Щоб отримати доступ до функцій модуля, його необхідно імпортувати на початку програми:

Імпорт модуля

```
import random
```

Основні функції модуля random

1. Дійсні числа: `random.random()` та `random.uniform()`

- ☑ `random.random()` — повертає випадкове дійсне число x у напіввідкритому інтервалі $[0.0, 1.0)$.
- ☑ `random.uniform(a, b)` — повертає випадкове дійсне число x у відрізку $[a, b]$.

Генерація дійсних чисел

```
import random
# Випадкове число від 0.0 до 1.0
p = random.random()
print("Імовірність",p)

# Випадкова температура в межах [-5.0, 15.0]
temp = random.uniform(-5.0, 15.0)
print("Температура:", temp)
```

2. Цілі числа: `random.randint()` та `random.randrange()`

- ☑ `random.randint(a, b)` — повертає випадкове ціле число $n \in [a, b]$ (включно з обома кінцями a та b).

- ☑ `random.randrange(start, stop, step)` — повертає випадковий елемент із множини цілих чисел із діапазону від start до stop-1 включно з кроком step.

Генерація цілих чисел

```
import random
# Моделювання кидка грального кубика (від 1 до 6)
dice = random.randint(1, 6)
print("Випало число:", dice)

# Випадкове парне число від 10 до 98
even_num = random.randrange(10, 100, 2)
print("Парне число:",even_num)
```

Відтворюваність результатів: `random.seed()`

У Python модуль `random` генерує не справжні випадкові числа, а псевдовипадкові. Це означає, що вони створюються за допомогою математичного алгоритму (зокрема, Вихор Мерсенна), який формує послідовність чисел, що виглядають випадковими, але насправді повністю визначаються початковим значенням — *seed*.

Якщо не задавати *seed*, генератор використовує системний час, і послідовність буде різною при кожному запуску програми.

Якщо задати `random.seed(42)`, то кожен запуск програми відтворюватиме одну й ту саму послідовність чисел. Це важливо для:

- ☑ **наукових експериментів** — щоб результати можна було перевірити й повторити;
- ☑ **тестування програм** — щоб відтворювати однакові сценарії;
- ☑ **навчання** — щоб приклади завжди давали однаковий результат.

Функція `random.seed(a)` фіксує початковий стан генератора, що є критично важливим для **відтворюваності результатів експерименту**.

Фіксація генератора

```
import random

random.seed(42)  # Задаємо фіксоване зерно
print(random.randint(1, 100))  # Завжди повертатиме один і той
                               # самий результат при повторних запуска
```

Контрольні питання

1. Для чого у Python використовується модуль `random`?
2. Які основні функції модуля `random` призначені для генерації дійсних чисел?
3. У чому різниця між `random.random()` та `random.uniform(a, b)`?
4. Які функції використовуються для генерації цілих чисел? Наведіть приклади.
5. Як працює функція `random.randint(a, b)`?
6. Як працює функція `random.randrange(start, stop, step)`?
7. Наведіть приклад використання генератора випадкових чисел для моделювання кидка кубика.
8. Як можна згенерувати випадкове парне число у заданому діапазоні?
9. Що означає поняття «псевдовипадкові числа»?
10. Яку роль відіграє функція `random.seed()`?
11. Чому важливо задавати `seed` у наукових експериментах та тестуванні програм?
12. Який алгоритм використовується у Python для генерації псевдовипадкових чисел?
13. Що відбувається, якщо не задавати `seed`? Як формується послідовність чисел?
14. Як можна забезпечити відтворюваність результатів при використанні модуля `random`?

Оператор циклу `for` та функція `range()`

7

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із заздалегідь відомою кількістю повторень є цикл `for`.

```
Синтаксис циклу for
for параметр in range(start, stop, step):
    блок_операторів
```

7.1 Генератор послідовностей `range()`

Найчастіше для формування послідовностей чисел у циклі `for` використовується вбудована функція `range()`. Вона може приймати від 1 до 3 аргументів:

- ☑ `range(stop)` — генерує числа від 0 до $stop - 1$ із кроком 1.
- ☑ `range(start, stop)` — генерує числа від $start$ до $stop - 1$ із кроком 1.
- ☑ `range(start, stop, step)` — генерує числа від $start$ до $stop - 1$ із кроком $step$.

Виклик `range()`	**Згенерована послідовність**	**Примітка**
`range(5)`	`0, 1, 2, 3, 4`	Від 0 до 4
`range(1, 6)`	`1, 2, 3, 4, 5`	Від 1 до 5
`range(2, 10, 2)`	`2, 4, 6, 8`	Парні числа від 2 до 8
`range(10, 0, -1)`	`10, 9, 8, 7, 6, 5, 4, 3, 2, 1`	Зворотний відлік

Таблиця 7.1: Приклади роботи функції `range()`

Якщо передати в range() лише одне число, за замовчуванням відлік починається з 0, а верхня межа не включається

```
for i in range(5):
    print(i)
```

Результат виведення : 0, 1, 2, 3, 4

range(start, stop) починає з числа start (включно) і зупиняється перед stop (не включаючи його).

```
for i in range(1,5):
    print(i)
```

Результат виведення: 1, 2, 3, 4

Третій параметр — це крок (step). range(1, 10, 2) починає з 1 і додає 2 на кожному кроці, генеруючи непарні числа до 10

```
for i in range(1,10,2):
    print(i)
```

Результат виведення: 1, 3, 5, 7, 9

Якщо крок від'ємний (-1), значення будуть зменшуватися. Цикл починається з 10 і зупиняється перед 0.

```
for i in range(10,0,-1):
    print(i)
```

Результат виведення: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

7.2 Базові алгоритми з циклом `for`

Приклад 1. Обчислення суми натуральних чисел від 1 до n

```
n = int(input("Введіть n "))
s = 0
for i in range(1, n + 1):
    s += i

print(f"Сума чисел від 1 до {n} дорівнює: {s}")
```

input(...) запитує у користувача число та приймає його як рядок (текст).

int(...) перетворює введений текст у ціле число та зберігає його в змінну n.

$s = 0$ створюється змінна-накопичувач s із початковим значенням 0. У ній буде зберігатися підсумкова сума.

for i in range(1, n + 1): цикл for перебирає числа по порядку.

range(1, n + 1) генерує послідовність від 1 до n включно (верхня межа $n+1$ не включається в діапазон, тому цикл зупиняється саме на n).

s += i на кожній ітерації циклу поточне число i додається до суми s (це скорочений запис для $s = s + i$).

print(f"Сума чисел від 1 до n дорівнює: s")

За допомогою f-рядка на екран виводиться підсумковий результат.

Приклад 2. Обчислення факторіала n!

```
n = int(input("Введіть n "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print(f"{n}! = {fact}")
```

Приклад 3. Табулювання функції y = sin(x) на відрізку [a, b]

```
import math

a = float(input("Введіть початок відрізка a "))
b = float(input("Введіть кінець відрізка b "))
n = 10
h = (b - a) / n

print(" x | sin(x)")
print("-------------")
for i in range(n + 1):
    x = a + i * h
    y = math.sin(x)
    print(f"{x:6.2f} | {y:6.4f}")
```

Форматоване виведення (виділений рядок):

$\{x : 6.2f\}$ — виводить значення x: займає 6 символів завширшки та округлюється до 2 знаків після коми. | — виводить розділювальну риску між стовпчиками.

$\{y : 6.4f\}$ — виводить значення y ($\sin(x)$): шириною в 6 символів та округлене до 4 знаків після коми.

Аналіз дільників числа та перевірка на простоту

Математична довідка

Просте число — це натуральне число, більше за 1, яке має точно два різні натуральні дільники: 1 та саме це число. Якщо число n має дільник, відмінний від 1 та n, то найменший такий дільник не перевищує $\lfloor\sqrt{n}\rfloor$.

Приклад 4. Виведення всіх дільників числа

```
n = int(input("Введіть натуральне число n "))
count = 0
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
        count += 1
print(f"Кількість дільників: {count}")
```

Пошук дільників: Програма перебирає всі цілі числа i від 1 до n включно і перевіряє умову `n % i == 0` (чи ділиться n на i без остачі).

Виведення та лічильник: Якщо число i є дільником: воно одразу виводиться на екран (`print(i)`); лічильник `count` збільшується на 1. Наприкінці програма виводить загальну кількість знайдених дільників.

Обчислення нескінченних математичних рядів

Обчислимо суму $S = \sum_{n=0}^{N} x^n = 1 + x + x^2 + \cdots + x^N$ ($|x| < 1$).

Приклад 5. Сума геометричного ряду

```
x = float(input("Введіть x (|x| < 1) "))
n = int(input("Введіть кількість членів N "))

s = 0.0
term = 1.0

for i in range(n):
    s += term
    term *= x

print(f"Сума ряду: {s:.6f}")
print(f"Точна сума 1/(1-x): {1 / (1 - x):.6f}")
```

Обчислення експоненти e^x через ряд Тейлора

Приклад 6. Обчислення e^x

```
import math

x = float(input("Введіть x "))
n = int(input("Кількість членів ряду "))

s = 0.0
term = 1.0

for i in range(1, n + 1):
    s += term
    term *= x / i

print(f"Наближене значення e^{x} = {s:.8f}")
print(f"Точне значення (math.exp): {math.exp(x):.8f}")
```

f"..."(f-рядок) — дозволяє вставляти значення змінних безпосередньо всередину тексту за допомогою фігурних дужок .

$\{s : .8f\}$ — бере значення змінної s (суму ряду) та форматує його як число з плавальною (рухомою) крапкою, залишаючи рівно 8 знаків після коми.

Цей рядок показує наближене значення експоненти e^x , вираховане через суму перших n членів ряду Тейлора.

Приклад 7. Таблиця множення від 2 до 10

```
print("Таблиця множення від 2 до 10")
print("====================================")

# Заголовок
for i in range(2, 11):
    print(f"{i:4}", end="")
print()
print("------------------------------------")

# Основна частина таблиці
for i in range(2, 11):           # зовнішній цикл — рядки
    for j in range(2, 11):       # внутрішній цикл — стовпчики
        print(f"{i*j:4}", end="")
    print()
```

Пояснення роботи вкладених циклів на прикладі таблиці множення

Зовнішній цикл (`for i in range(2, 11):`) відповідає за формування рядків таблиці. На кожній ітерації він бере нове значення i (від 2 до 10) і запускає внутрішній цикл.

Внутрішній цикл (`for j in range(2, 11):`) відповідає за стовпчики таблиці. Для кожного значення j (від 2 до 10) він обчислює добуток $i \cdot j$ і виводить його на екран.

Комбінація циклів:

- ☑ Коли внутрішній цикл завершує всі ітерації для певного i, виконується `print()` без аргументів — це перехід на новий рядок.
- ☑ Потім зовнішній цикл переходить до наступного значення i, і знову запускається внутрішній цикл.
- ☑ Таким чином формується таблиця: кожен рядок відповідає одному множнику i, а кожен стовпчик — множнику j.

Приклад роботи:

- ☑ При $i = 2$ внутрішній цикл обчислює: $2 \cdot 2, 2 \cdot 3, \ldots, 2 \cdot 10$.
- ☑ При $i = 3$ внутрішній цикл обчислює: $3 \cdot 2, 3 \cdot 3, \ldots, 3 \cdot 10$.
- ☑ І так далі до $i = 10$.

У результаті отримуємо повну таблицю множення, де **зовнішній цикл керує рядками**, а **внутрішній — стовпчиками**.

7.3 Керування циклом: `break`, `continue` та `else`

- ☑ `break` — достроково припиняє виконання циклу.
- ☑ `continue` — пропускає залишок тіла циклу та переходить до наступної ітерації.
- ☑ `else` — блок коду, який виконується після завершення циклу, якщо вихід з нього **не** відбувся за допомогою `break`.

Приклад 8. Пошук першого від'ємного числа з використанням break та else

```
for i in range(5):
    val = int(input("Введіть число "))
    if val < 0:
        print(f"Знайдено перше від'ємне число: {val}")
        break
else:
    print("Усі введені числа є невід'ємними.")
```

Контрольні питання

1. Який синтаксис циклу `for` у Python?
2. Які параметри може приймати функція `range()`? Поясніть їх призначення.
3. Що відбувається, якщо у `range()` передати лише один аргумент?
4. Як працює `range(start, stop)`? Які межі включаються, а які — ні?
5. Як використати третій параметр `step` у функції `range()`?
6. Що відбувається, якщо параметр `step` є від'ємним?
7. Як за допомогою циклу `for` обчислити суму чисел від 1 до n?
8. Як за допомогою циклу `for` обчислити факторіал числа n?
9. Як можна використати цикл `for` для табулювання функції (наприклад, $y = \sin(x)$)?
10. Як працюють вкладені цикли? Наведіть приклад побудови таблиці множення.
11. У чому полягає різниця між зовнішнім та внутрішнім циклом у вкладеній структурі?
12. Як працюють оператори `break`, `continue` та `else` у циклі `for`?

Практичні завдання

1. Надрукувати числа від 0 до 9 у рядок через пробіл.
2. Надрукувати числа від 0 до 9 п'ять разів.
3. Надрукувати матрицю (таблицю) 5×5, де в першому рядку нулі, у другому — одиниці, і так далі. Потім переписати код так, щоб кількість рядків та стовпців вводив користувач.
4. Надрукувати таблицю множення від 2 до 10.

5. Надрукувати матрицю 5×5, елементи якої — зірочки, а при $i = j$ надрукувати символ \.
6. Запитати в користувача номер рядка r та стовпця c, у якому буде символ `*`, всі інші елементи — нулі.
7. Обчислити суму чисел від 1 до n, використовуючи цикл `for`.
8. Обчислити факторіал числа n.
9. Табулювати функцію $y = \sin(x)$ на відрізку $[a, b]$ з кроком h.
10. Вивести всі дільники числа n та їх кількість.
11. Обчислити суму перших N членів геометричного ряду $1 + x + x^2 + \ldots$ при $|x| < 1$.
12. Обчислити наближене значення e^x за допомогою ряду Тейлора.

Оператор циклу `while`

8

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із умовою повторення (циклом із передумовою) є цикл `while`.

```
Синтаксис циклу while
while вираз_умови:
    блок_операторів
```

Блок операторів циклу `while` повторюється до того часу, поки результатом логічного висловлювання умови повтору є значення `True`. Вираз умови обчислюється перед кожним повтором циклу, і вихід з циклу (тобто перехід до оператора, наступного за оператором `while`) здійснюється в тому випадку, якщо вираз умови має значення `False`.

Приклад 1. Виведення значень лічильника від 0 до 4

```
i = 0
while i < 5:
    print("значення лічильника =", i)
    i += 1  # i = i + 1

print("end")
```

Покроковий аналіз виконання:

Задається початкове значення змінної $i = 0$.

Перевіряється умова $i < 5$. Оскільки вона виконується (`True`), відбувається перехід до операторів всередині циклу (виводимо значення i на екран та збільшуємо на 1, тобто $i = 0 + 1 = 1$).

Знову повертаємось до перевірки умови на початку $i < 5$. Вона виконується $\rightarrow$ виконуються оператори виводу на екран та збільшення i, і так до тих пір, поки значення i не стане дорівнювати 5. Тоді оператори всередині циклу вже не будуть виконуватись, виконання перейде до оператора `print("end")`.

Різниця між циклами for та while

- **Цикл for** використовується тоді, коли кількість повторень відома наперед. Він перебирає елементи послідовності (наприклад, `range()`, список, рядок) і виконує тіло циклу для кожного елемента.
- **Цикл while** застосовується тоді, коли кількість повторень невідома заздалегідь. Він виконує тіло циклу доти, доки умова залишається істинною (`True`). Якщо умова стає хибною (`False`), цикл завершується.

8.1 Керування циклом: `break`, `continue`, `pass` та блок `else`

```
Структура керування циклом while
while вираз_умови_test:
    блок_операторів_1
    if умова_1:
        break     # вийти з циклу, пропустивши else
    if умова_2:
        continue  # перейти на початок циклу до
                  #виразу_умови_test
    блок_операторів_2
else:
    оператори_else # виконуються, якщо вираз_умови
                   # став False (без break)
```

- **`break`** — вихід з циклу із пропуском блоку `else`.
- **`continue`** — перехід на початок циклу до перевірки умови.
- **`pass`** — порожня інструкція. Не виконує жодних дій і використовується у випадках, коли синтаксис мови потребує наявності інструкції, але поки що жодних корисних дій виконати не потрібно:

```
if True:
    pass  # інструкції будуть написані пізніше

while True:
```

```
    pass  # порожній нескінченний цикл
```

8.2 Типові застосування циклу `while`

Приклад 2. Інтерактивна перевірка умов (перевірка стану)

```
x = input("Двері відчинені? ")
while x.lower() != "так":
    print("стукаємо")
    x = input("Двері відчинені? ")
print("Заходимо")
```

Примітка: Блок операторів у `while` може не виконатись жодного разу, якщо вираз-умова від самого початку має значення `False`.

Приклад 3. Табулювання функції y = sin(x) до першого від'ємного значення

```
import math

x = 0.0
while math.sin(x) >= 0:
    print(f"sin({x:.1f}) = {math.sin(x):.4f}")
    x += 0.1
```

Приклад 4. Запит на введення додатного числа з валідацією

```
x = int(input("Введіть додатне x = "))
while x <= 0:
    x = int(input("Введіть додатне x = "))
print("Дякую")
```

Програма буде виводити на екран повідомлення та перевіряти, яке число ввів користувач. Якщо $x \leq 0$, то повідомлення із запрошенням до введення буде з'являтися знову й знову. Якщо число додатне, то цикл завершиться, бо умова $x \leq 0$ перестане виконуватись, та виконається рядок коду, що йде одразу після циклу while.

Приклад 5. Обчислення суми введених чисел до введення 0 (стоп-сигнал)

```
s = 0.0
x = float(input())
while x != 0:
    s += x
    x = float(input())
print("Сума =", s)
```

Ініціалізується змінна s, яка буде накопичувати суму введених чисел.

Користувач вводить число з клавіатури, яке перетворюється у тип float (дійсне число).

Запускається цикл, який працює доти, доки введене число не дорівнює нулю.

До змінної s додається введене число x.

Користувач вводить наступне число. Якщо це знову не нуль — цикл повторюється.

Коли користувач вводить 0, цикл завершується, і програма виводить загальну суму всіх введених чисел.

Приклад 6. Обчислення суми нескінченного ряду (розклад exp(x)) з точністю eps

```
# Розклад e^x = 1 + x + x^2/2! + ... + x^n/n! + ...
import math
# Початкові дані та параметри
x = 1.0        # Значення x, для якого обчислюємо e^x
eps = 1e-9     # Точність обчислень (10^-9)
s = 0.0        # Змінна для накопичення суми ряду
term = 1.0     # Перший доданок ряду (при n = 0, x^0 / 0! = 1)
i = 1          # Лічильник ітерацій та порядковий номер
#наступного доданка

# Цикл виконується, поки модуль поточного доданка більший за
#задану точність eps
while abs(term) > eps:
    s += term            # Додаємо поточний доданок до  суми
    term = term * x / i # Обчислюємо наступний доданок
    i += 1               # Збільшуємо номер ітерації на 1
print(f"Сума s = {s}")
print(f"Ітерацій: {i - 1}")
```

Примітка: Будь-який цикл `for` можна записати за допомогою `while`, але зво-

ротне твердження, взагалі кажучи, є хибним, оскільки `while` призначений для циклів з невідомою заздалегідь кількістю повторень.

8.3 Проблема «зациклювання»

Іноді при неуважному використанні оператора `while` виникає так зване **зациклювання** — нескінченне повторення тіла циклу. Це може статися з двох основних причин:

1. Змінні, що є частиною висловлювання умови циклу, не змінюють усередині циклу своїх значень або набувають некоректних значень, у результаті чого вираз умови завжди має значення `True`.
2. Некоректним є сам вираз умови циклу (наприклад, порівняння дійсних чисел типу `float` на строгу рівність).

Приклад зациклювання 1: Нескінченний цикл при перериванні Ctrl+C

```
while True:
    print("Натисніть Ctrl+C для зупинки")
```

while True: — створює нескінченний цикл. Оскільки умова True (істина) завжди залишається True і ніколи не змінюється на False, цикл буде виконуватися вічно.

print(”Натисніть Ctrl+C для зупинки”) — тіло циклу, яке безперервно виводить це повідомлення в консоль/термінал.

Зациклювання виникає тому, що у коді немає жодної умови виходу (наприклад, оператора break або зміни логічного виразу), тому програма самостійно зупинитися не може.

Щоб примусово перервати виконання нескінченного циклу в терміналі чи консолі, використовують комбінацію клавіш Ctrl + C. Вона надсилає програмі сигнал переривання (KeyboardInterrupt), що викликає зупинку виконання коду.

Приклад зациклювання 2: Небезпечне порівняння дійсних чисел (float)

```
# Некоректний код
x = 0.0
while x != 1.0:# умова не перестане виконуватись ніколи
    print(f"x = {x:.1f}")
    x += 0.3

# Правильний варіант:
x = 0.0
while x < 1.0:
    print(f"x = {x:.1f}")
    x += 0.3
```

8.4 Складніші приклади та алгоритми

Приклад 7. Середнє арифметичне простих чисел до введення 0

```
x = int(input())  # Зчитуємо перше ціле число з клавіатури
k = 0             # Лічильник знайдених простих чисел
S = 0             # Змінна для збереження суми простих чисел

# Цикл триває, поки користувач не введе 0
# ( 0 - ознака кінця введення)
while x != 0:
    is_prime = True  # Прапорець: припускаємо,
                     #  що число x є простим
    if x <= 1:
        is_prime = False  # Числа <= 1 не є простими
    else:
        # Наступний цикл перевіряє наявність дільників
        # від 2 до кореня з x включно
        for i in range(2, int(x**0.5) + 1):
            if x % i == 0: # Якщо x ділиться на i без остачі
                is_prime = False  # то воно не просте, і далі
                break             # можна не перевіряти

    # Якщо число виявилося простим
    if is_prime:
        k += 1  # Збільшуємо кількість простих чисел на 1
        S += x  # Додаємо число до загальної суми

    x = int(input())  # Зчитуємо наступне число для наступної
                      # ітерації циклу

# Підбиття підсумків після виходу з циклу
if k == 0: # Якщо жодного простого числа не було введено
    print("Простих чисел немає")
else:
    # Обчислюємо та виводимо середнє арифметичне (сума/кількість)
    print("Середнє арифметичне простих:", S / k)
```

Програма почергово приймає цілі числа, поки не буде введено 0 (сигнал зупинки). Для кожного введеного числа перевіряється, чи є воно простим (тобто більшим за 1 і ділиться лише на 1 та саме себе). Якщо число виявилося простим, програма збільшує лічильник k на 1 та додає це число до загальної суми S. Після введення

0 програма ділить суму простих чисел на їхню кількість (S / k) або повідомляє, що простих чисел не було введено.

Приклад 8. Перевірка наявності цифри 4 у натуральному числі

```
N = int(input('N = '))  # Зчитуємо натуральне число
temp = N                # Створюємо тимчасову копію числа,
# щоб не змінювати оригінальне N

# Цикл триває, поки у тимчасовій змінній залишаються цифри
#(число більше 0)
while temp > 0:
    x = temp % 10   # Отримуємо останню цифру числа
                    #(остача від ділення на 10)

    if x == 4:      # Перевіряємо, чи дорівнює ця цифра 4
        print('Є 4')#Виводимо повідомлення, що цифру 4 знайдено
        break   # Достроково перериваємо цикл,
                # оскільки перевіряти далі немає потреби
    #Відкидаємо останню цифру
    temp //= 10  # (цілочислове ділення на 10)

else: # Блок else виконується лише тоді,
   # коли цикл while завершився природним шляхом
   # (тобто temp стало <= 0), і не був перерваний break
   print('Немає 4')
```

Користувач вводить число N. Програма створює його копію temp, щоб зберегти оригінальне значення. Отримує останню цифру числа за допомогою temp % 10. Якщо цифра дорівнює 4, виводиться ’Є 4’, і цикл негайно зупиняється (break). Якщо цифра не 4, вона відкидається за допомогою цілочислового ділення temp на 10. Якщо цикл закінчився і цифру 4 так і не знайшли, спрацьовує блок else, який виводить ’Немає 4’.

Приклад 9. Алгоритм Евкліда пошуку найбільшого спільного дільника (НСД)

```
a = int(input("a = "))
b = int(input("b = "))

while a > 0 and b > 0:
    if a > b:
        a %= b
    else:
        b %= a

print("НСД =", a + b)
```

Контрольні питання

1. Який синтаксис циклу `while` у Python?
2. У чому полягає відмінність між циклами `for` та `while`?
3. Коли доцільно використовувати цикл `while`, а коли цикл `for`?
4. Що означає «цикл із передумовою»?
5. Що відбувається, якщо умова циклу `while` від самого початку має значення `False`?
6. Як працюють оператори `break`, `continue`, `pass` у циклі `while`?
7. Для чого використовується блок `else` у циклі `while`?
8. Що таке «зациклювання»? Які основні причини його виникнення?
9. Як уникнути нескінченного циклу у програмі?
10. Наведіть приклад задачі, де цикл `while` зручніший за цикл `for`.

Практичні завдання

1. Надрукувати числа від 0 до 9 за допомогою циклу `while`.
2. Обчислити суму чисел від 1 до n, використовуючи цикл `while`.
3. Запитувати у користувача пароль доти, доки він не введе правильний.
4. Вивести таблицю квадратів чисел від 1 до 10.
5. Реалізувати програму, яка знаходить найбільший спільний дільник двох чисел (алгоритм Евкліда).
6. Написати програму, яка виводить усі цифри введеного числа по одній.

7. Обчислити факторіал числа n за допомогою циклу `while`.
8. Запитувати у користувача додатне число доти, доки він не введе коректне значення.
9. Обчислити середнє арифметичне простих чисел, що вводяться користувачем, до введення нуля.
10. Реалізувати програму, яка перевіряє, чи містить введене число цифру нуль.
11. Обчислити суму введених чисел до введення нуля (стоп□сигнал).
12. Написати нескінченний цикл `while True`, який виводить повідомлення, і перервати його комбінацією клавіш `Ctrl+C`.

Бібліографія

[1] Васильєв О. *Програмування мовою Python*. – Тернопіль: НК «Богдан», 2019. – 504 с.

[2] Руденко В., Жугастров О. *Інформатика. Основи алгоритмізації та програмування мовою Python*. – Харків: Ранок, 2019. – 192 с.

[3] Кормен Т., Лейзерсон Ч., Ривест Р. *Вступ до алгоритмів* / Т. Кормен, Ч. Лейзерсон, Р. Ривест. – Київ: К.І.С., 2019. – 1288 с.

[4] Матвієнко М. П. *Теорія алгоритмів*. – Київ: Видавництво Ліра-К, 2019. – 344 с.

[5] Бородкіна І. Л., Бородкін Г. О. *Теорія алгоритмів. Посібник для студентів вищих навчальних закладів* / І. Л. Бородкіна, Г. О. Бородкін. – Київ: Центр навчальної літератури, 2019. – 184 с.

[6] Матвієнко М. П., Шаповалов С. П. *Математична логіка та теорія алгоритмів*. – Київ: Центр навчальної літератури, 2015. – 212 с.

Страхов Євген Михайлович
Платонова Євгенія Вікторівна

Основи програмування мовою Python. Структурне програмування.

Конспект лекцій

для здобувачів спеціальності Е7 Математика

Електронне мережне навчальне видання

В авторській редакції

Затвердж. авт. . Шрифт Times New Roman.
Системні вимоги: операційна система сумісна з програмним забезпеченням для читання файлів формату PDF.
Обсяг 921 Кб. Зам. № .

Видавець і виготовлювач:
Одеський національний університет імені І. І. Мечникова
Свідоцтво суб'єкта видавничої справи ДК № 4215 від 22.11.2011 р.
вул. Університетська, 12, м. Одеса, 65082, Україна
Тел.: (048) 723 28 39, e-mail: druk@onu.edu.ua